Initial State

buffer = []
size = 0

2 threads start:

thread t1(producer);
thread t2(consumer);

STEP 1 → Producer starts

unique_lock<mutex> lock(mtx);

Producer ne mutex lock kar diya.

Ab consumer queue touch nahi kar sakta.

STEP 2 → Buffer full check

while(buffer.size()==BUFFER_MAX)

Check:

0 == 5 ? NO

to wait nahi karega.

STEP 3 → Producer pushes item

buffer.push(1);

Now:

buffer = [1]

Print:

Producer: item 1 dala

STEP 4 → notify consumer

cv.notify_one();

Agar consumer so raha hoga to uth jayega.

STEP 5 → iteration khatam

Loop iteration end.

lock destroy

Mutex auto unlock.

STEP 6 → Consumer gets CPU

Consumer:

unique_lock<mutex> lock(mtx);

mutex lock.

STEP 7 → Empty check

while(buffer.empty())

Check:

buffer empty ? NO

STEP 8 → Consumer pops

item = buffer.front();
buffer.pop();

Item = 1

Now:

buffer = []

Print:

Consumer: Item 1 nikala

STEP 9 → notify producer

cv.notify_one();

Agar producer so raha hoga to uth jayega.

STEP 10 → unlock

Iteration end → lock destroy → mutex unlock.

Ab same cycle repeat hota rahega

Producer:

2 push

3 push
4 push
5 push

Consumer:

2 pop
3 pop
4 pop

Depends scheduler pe.

IMPORTANT CASE → Buffer FULL

Suppose producer fast hai.

State:

buffer = [1 2 3 4 5]
size = 5

Producer next iteration:

while(buffer.size()==BUFFER_MAX)

Check:

5 == 5 YES

Now:

cv.wait(lock);

---

wait() internally kya karta hai

BEFORE WAIT

producer holds mutex

---

wait() called

1. mutex unlock
2. producer sleep

Ab producer sota hai.

Consumer enters now

Consumer mutex le lega.

buffer.pop();

Now:

buffer = [2 3 4 5]

size = 4

Consumer notify

cv.notify_one();

Producer wake up.

Producer wake hone ke baad

`wait()` se bahar aane se pehle:

mutex dubara lock karega

Then:

buffer.push(6);

IMPORTANT CASE → Buffer EMPTY

Suppose consumer fast hai.

State:

buffer = []

Consumer:

while(buffer.empty())

YES.

Then:

cv.wait(lock);

Consumer:

mutex unlock
sleep

Producer inserts item

buffer.push(7);

Then:

cv.notify_one();

Consumer wake up.

WHY NO DEADLOCK?

Because:

cv.wait(lock);

mutex release karta hai.

Agar release nahi karta:

- producer bhi wait
- consumer bhi wait
- deadlock

MOST IMPORTANT TIMELINE

Producer lock
Producer push
Producer unlock

Consumer lock
Consumer pop
Consumer unlock

---

wait() timeline

Producer:
lock
buffer full
wait()

wait():
unlock mutex
sleep

Consumer:
lock

pop
notify

Producer:
wake
re-lock mutex
continue

Real meaning of synchronization

Without mutex:

Producer and Consumer same time queue modify karte

Result:

- corruption
- crash
- undefined behavior

Final Output Approx

Producer: item 1 dala
Consumer: item 1 nikala

Producer: item 2 dala
Producer: item 3 dala

Consumer: item 2 nikala
Consumer: item 3 nikala

Order exact fixed nahi hota because threads concurrent hain.

Sabse Important 4 lines

Lock

unique_lock<mutex> lock(mtx);

Sleep

cv.wait(lock);

Wake

cv.notify_one();

Shared resource

queue<int> buffer;